

리눅스 명령어 & 패키지 정리

grep · fdfind(fd) · fdupes · rdfind

1. grep

grep은 파일이나 입력 스트림에서 특정 패턴(문자열, 정규표현식)을 검색하는 리눅스/유닉스 명령어입니다.

1.1 기본 문법

```
grep [옵션] "검색패턴" 파일명
```

1.2 기본 검색

```
# 파일에서 "error" 문자열이 포함된 줄 찾기
grep "error" app.log

# 예시 출력
# 2026-08-11 10:23:01 error: connection timeout
# 2026-08-11 10:25:44 error: invalid token
```

1.3 자주 쓰는 옵션들

-i : 대소문자 구분 없이 검색

```
grep -i "error" app.log
# ERROR, Error, error 모두 찾음
```

-n : 줄 번호 표시

```
grep -n "error" app.log
# 42:2026-08-11 10:23:01 error: connection timeout
```

-r (또는 -R) : 디렉터리 재귀 검색

```
# src 디렉터리 전체에서 "TODO" 검색
grep -r "TODO" ./src

# 예시 출력
# ./src/utils.py:15: # TODO: 예외처리 추가
```

```
# ./src/main.py:88: # TODO: 리팩토링 필요
```

-v : 패턴이 없는 줄만 출력 (반전 검색)

```
# "DEBUG"가 포함되지 않은 줄만 보기  
grep -v "DEBUG" app.log
```

-c : 일치하는 줄의 개수만 출력

```
grep -c "error" app.log  
# 17
```

-l / -L : 파일명만 출력

```
# 패턴이 있는 파일명만  
grep -rl "import pandas" ./src  
  
# 패턴이 없는 파일명만  
grep -rL "import pandas" ./src
```

-w : 단어 단위로 정확히 일치

```
# "cat"만 찾고 "category", "concatenate" 등은 제외  
grep -w "cat" animals.txt
```

-A, -B, -C : 앞뒤 문맥 함께 출력

```
# 일치한 줄 뒤로 3줄 더 출력 (After)  
grep -A 3 "Traceback" error.log  
  
# 일치한 줄 앞으로 2줄 출력 (Before)  
grep -B 2 "Traceback" error.log  
  
# 앞뒤로 각각 2줄씩 출력 (Context)  
grep -C 2 "Traceback" error.log
```

1.4 정규표현식 활용

-E : 확장 정규표현식 (Extended regex)

```
# "error" 또는 "warning"이 포함된 줄  
grep -E "error|warning" app.log  
  
# 이메일 형식 찾기
```

```
grep -E "[a-zA-Z0-9._%+ -]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}" contacts.txt
```

자주 쓰는 정규식 패턴 예시

```
# 숫자로 시작하는 줄
grep -E "^[0-9]" data.txt

# IP 주소 패턴 찾기
grep -E "([0-9]{1,3}\\.[0-9]{1,3})" access.log

# "test"로 시작하고 ".py"로 끝나는 줄
grep -E "^test.*\\.py$" filelist.txt
```

1.5 실전 활용 예제

예제 1: 특정 프로세스 찾기

```
ps aux | grep "python"
# 실행 중인 python 프로세스만 필터링
```

예제 2: 여러 파일 확장자에서 검색

```
grep -r --include="*.py" "def calculate" ./project
```

예제 3: 특정 패턴 제외하고 검색

```
grep -r --exclude="*.log" "TODO" ./project
```

예제 4: 여러 패턴을 파일로 관리해서 검색

```
# patterns.txt 안에 검색할 단어들을 줄바꿈으로 나열
grep -f patterns.txt data.txt
```

예제 5: CSV 파일에서 특정 컬럼 값 찾기

```
# GD RDM 프로젝트 CSV처럼 컬럼이 많은 데이터에서 특정 값 필터링
grep "FAIL" diagnostic_data.csv | head -20
```

예제 6: git 과 함께 사용

```
# 특정 함수를 사용하는 커밋 이력 검색
git log -p | grep -n "deprecated_function"
```

1.6 옵션 조합 예시 (실무에서 자주 쓰는 패턴)

```
# 대소문자 구분 없이, 줄 번호와 함께, 하위 디렉터리 전체 검색
```

```
grep -rin "connection refused" ./logs

# 정확한 단어 매칭 + 앞뒤 문맥 확인
grep -w -C 2 "timeout" server.log

# 개수만 세면서 재귀 검색
grep -rc "import numpy" ./src | grep -v ":0"
```

1.7 grep vs egrep vs fgrep

명령어	설명
grep	기본 정규표현식(BRE)
grep -E (= egrep)	확장 정규표현식(ERE), , +, ? 등을 이스케이프 없이 사용
grep -F (= fgrep)	정규식 아닌 순수 문자열 검색 (속도 빠름)

```
# 특수문자가 포함된 문자열을 그대로 검색할 때
grep -F "a.b+c" text.txt
```

2. fdfind (fd)

fdfind(fd)는 find의 대체 도구로, 더 빠르고 직관적인 문법을 가진 파일/디렉터리 검색 프로그램입니다.

2.0 명령어 이름 참고

Ubuntu/Debian 계열에서는 fd라는 이름이 다른 패키지와 충돌해서 fdfind로 설치됩니다.

```
# 별칭 설정 (선택사항)
alias fd='fdfind'

# 또는 심볼릭 링크 생성
mkdir -p ~/.local/bin
ln -s $(which fdfind) ~/.local/bin/fd
```

이 가이드에서는 fdfind로 통일해서 설명합니다.

2.1 기본 문법

```
fdfind [옵션] [패턴] [검색경로]
```

경로를 생략하면 현재 디렉터리부터 재귀적으로 검색합니다.

2.2 기본 검색

```
# 현재 디렉터리에서 "config"가 포함된 파일/디렉터리 찾기
fdfind config

# 예시 출력
# ./src/config.py
# ./config/settings.yaml
# ./tests/config_test.py
```

grep과 달리 기본적으로 .gitignore, 숨김 파일, node_modules 등을 자동으로 제외합니다.

2.3 자주 쓰는 옵션들

-e (--extension) : 확장자로 검색

```
# 모든 .py 파일 찾기
fdfind -e py
```

```
# 여러 확장자 동시 검색
fdfind -e py -e ipynb
```

-t (--type) : 타입으로 검색

```
fdfind -t f      # 파일(file)만
fdfind -t d      # 디렉터리(directory)만
fdfind -t l      # 심볼릭 링크(symlink)만
fdfind -t x      # 실행 가능한 파일만
```

-H (--hidden) : 숨김 파일 포함

```
# .env, .gitignore 같은 숨김 파일도 검색
fdfind -H ".env"
```

-I (--no-ignore) : .gitignore 무시하고 검색

```
# node_modules, .gitignore 규칙 무시하고 전체 검색
fdfind -I "test"
```

-i / -s : 대소문자 옵션

```
fdfind -i readme  # 대소문자 무시 (기본값이 스마트 케이스라 보통 자동 적용됨)
fdfind -s README  # 대소문자 구분 강제
```

-d (--max-depth) : 검색 깊이 제한

```
# 현재 디렉터리 기준 2단계까지만 검색
fdfind -d 2 ".json"
```

-x / -X : 검색 결과에 명령 실행

```
# 찾은 모든 .py 파일에 대해 black 포맷터 실행
fdfind -e py -x black {}

# 찾은 결과를 한 번에 모아서 명령 실행 (X는 대문자, 배치 실행)
fdfind -e log -X rm
```

2.4 정규표현식 vs 글롭(glob) 검색

기본은 정규표현식

```
# "test"로 시작하는 파일
fdfind "^test"
```

```
# 숫자가 포함된 파일명
fdfind "[0-9]+"
```

-g (--glob) : 글롭 패턴 사용

```
# 셸 와일드카드 스타일로 검색
fdfind -g "*.csv"
fdfind -g "diagnostic_*.csv"
```

2.5 실전 활용 예제

예제 1: 특정 확장자 파일을 크기순 정렬해서 확인

```
fdfind -e csv -x ls -lh {} | sort -k5 -h
```

예제 2: GD RDM 프로젝트처럼 대용량 CSV 파일 찾기

```
# 542컬럼 스키마 관련 CSV 파일 검색
fdfind -e csv "diagnostic" ./data
```

예제 3: 특정 디렉터리 제외하고 검색

```
fdfind -E "node_modules" -E "__pycache__" -e py
```

예제 4: 최근 수정된 파일 찾기

```
# 최근 1일 이내 수정된 파일
fdfind --changed-within 1d

# 최근 2주 이내 수정된 파일
fdfind --changed-within 2weeks

# 특정 시점 이전 파일
fdfind --changed-before 2026-01-01
```

예제 5: 파일 크기로 필터링

```
# 10MB보다 큰 파일 찾기
fdfind -S +10M

# 1KB보다 작은 파일 찾기
fdfind -S -1k
```

예제 6: 찾은 결과를 grep 과 파이프 연결

```
# .py 파일 중 "import torch"가 포함된 파일만 최종 필터링
fdfind -e py -X grep -l "import torch"
```

예제 7: 절대 경로로 출력

```
fdfind -a "₩.env"
```

2.6 옵션 조합 예시 (실무에서 자주 쓰는 패턴)

```
# 숨김 파일 포함, .gitignore 무시, 확장자 필터, 특정 디렉터리 제외
fdfind -H -l -e py -E venv -E __pycache__ "test_"

# 최근 수정된 설정 파일만 찾아서 백업
fdfind -e yaml -e json --changed-within 1d -x cp {} ./backup/
```

2.7 find 대비 fdfind 의 장점

항목	find	fdfind
기본 검색 속도	느림	병렬 처리로 빠름
문법	복잡 (-name, -iname 등)	직관적
기본 제외 규칙	없음	.gitignore, 숨김파일 자동 제외
색상 출력	없음	기본 지원
정규표현식	기본 미지원	기본 지원

3. fdupes

fdupes는 중복 파일을 찾아주는 도구입니다.

3.1 기본 문법

```
fdupes [옵션] 디렉터리
```

3.2 설치

```
sudo apt install fdupes
```

3.3 기본 사용

```
# 현재 디렉터리에서 중복 파일 찾기 (비재귀)
fdupes .

# 예시 출력 (중복 그룹별로 빈 줄로 구분되어 출력됨)
# ./photos/image1.jpg
# ./backup/image1_copy.jpg
#
# ./docs/report.pdf
# ./old_docs/report.pdf
```

3.4 자주 쓰는 옵션들

-r (--recurse) : 하위 디렉터리까지 재귀 검색

```
fdupes -r ./data
```

-R : 특정 인자 이후 디렉터리만 재귀 검색

```
# ./keep는 비재귀, ./scan은 재귀 검색
fdupes -R ./scan ./keep
```

-n (--noempty) : 빈 파일(0 바이트) 제외

```
fdupes -rn ./data
```

-s (--symlinks) : 심볼릭 링크도 검사 대상에 포함

```
fdupes -rs ./data
```

-q (--quiet) : 진행 상태 메시지 숨김

```
fdupes -rq ./data
```

-f (--omitfirst) : 각 그룹의 첫 번째 파일은 목록에서 제외

```
# 삭제할 목록만 뽑을 때 유용 (원본은 남기고 나머지만 출력)
fdupes -rf ./data
```

3.5 실전 활용 예제

예제 1: 중복 파일 개수/용량 요약

```
fdupes -r -m ./data
# 예시 출력: 128 duplicate files (in 42 sets), occupying 3.4 megabytes
```

예제 2: 중복 파일을 대화형으로 골라서 삭제

```
fdupes -r -d ./data
# 각 그룹마다 어떤 파일을 남길지 번호로 선택하는 프롬프트가 뜸
```

△ 주의: -d 옵션은 실수로 원본까지 삭제될 위험이 있으니 반드시 백업 후 사용해야 합니다.

예제 3: 첫 번째 파일 자동으로 보존하고 나머지 자동 삭제

```
fdupes -r -d -N ./data
# -N: 확인 없이 각 그룹의 첫 번째 파일을 자동으로 남기고 나머지 삭제
```

예제 4: 중복 파일 목록만 파일로 저장 (삭제는 하지 않음)

```
fdupes -rf ./data > duplicates_to_review.txt
```

예제 5: 크기순으로 정렬해서 출력

```
fdupes -r --order=size ./data
```

예제 6: 하드링크로 변환해서 용량 절약 (삭제 없이 중복 제거 효과)

```
fdupes -r -L ./data
```

4. rfind

rfind 역시 중복 파일을 찾아주는 도구입니다.

4.1 기본 문법

```
rfind [옵션] 디렉터리1 [디렉터리2 ...]
```

4.2 설치

```
sudo apt install rfind
```

4.3 기본 사용

```
rfind ./data
```

실행 후 results.txt 파일이 생성됨 (기본 동작)

```
cat results.txt
```

rfind는 기본적으로 재귀 검색이며, 여러 디렉터리를 동시에 지정할 수 있습니다.

4.4 자주 쓰는 옵션들

-dryrun : 실제 변경 없이 시뮬레이션만 수행

```
rfind -dryrun true ./data
```

-removeidentinode : 동일 inode(하드링크) 파일을 중복으로 취급할지 여부

```
rfind -removeidentinode true ./data
```

-ignoreempty : 빈 파일 무시

```
rfind -ignoreempty true ./data
```

-minsize : 최소 크기 이하 파일 제외

1KB 미만 파일은 검사에서 제외

```
rfind -minsize 1024 ./data
```

-deleteduplicates : 중복 파일 실제 삭제

```
rfind -deleteduplicates true ./data
```

-makesymlinks : 중복 파일을 심볼릭 링크로 치환

```
rfind -makesymlinks true ./data
```

-makehardlinks : 중복 파일을 하드링크로 치환 (용량 절약)

```
rdfind -makehardlinks true ./data
```

-outputname : 결과 파일명 지정

```
rdfind -outputname scan_result.txt ./data
```

4.5 우선순위 지정 (어떤 파일을 "원본"으로 볼지)

rdfind는 여러 디렉터리를 순서대로 지정하면, 먼저 지정한 디렉터리의 파일을 원본(우선순위 높음)으로 취급합니다.

```
# ./originals에 있는 파일을 원본으로 간주, ./backup의 동일 파일을 중복으로 판단
rdfind ./originals ./backup
```

4.6 실전 활용 예제

예제 1: 안전하게 먼저 결과만 확인

```
rdfind -dryrun true ./data
cat results.txt
```

예제 2: 결과 확인 후 실제로 중복 삭제

```
rdfind -deleteduplicates true ./data
```

예제 3: 대용량 백업 폴더에서 용량 절약을 위해 하드링크로 전환

```
rdfind -makehardlinks true ./backup_2025 ./backup_2026
```

예제 4: 특정 크기 이상 파일만 검사 (사진/동영상처럼 큰 파일 위주)

```
rdfind -minsize 1000000 ./photos
```

예제 5: 여러 옵션 조합 (안전한 정리 워크플로우)

```
rdfind -ignoreempty true -minsize 1024 -dryrun true ./data
# 결과 확인 후 -dryrun false로 재실행하여 실제 삭제/링크 처리
```

5. fdupes vs rfind 비교

항목	fdupes	rfind
결과 확인 방식	표준 출력(그룹별)	results.txt 파일로 저장
삭제 방식	대화형(-d) 또는 자동(-N)	옵션으로 일괄 삭제(-deleteduplicates)
하드링크 변환	지원 (-L)	지원 (-makehardlinks)
심볼릭링크 변환	미지원	지원 (-makesymlinks)
우선순위(원본) 지정	제한적	디렉터리 순서로 명확하게 지정 가능
dry-run(시뮬레이션)	없음 (직접 확인 필요)	지원 (-dryrun true)
대용량 처리 성능	준수	해시 비교 방식으로 대용량에 유리

5.1 실무 추천 워크플로우

```
# 1단계: rfind로 먼저 안전하게 시뮬레이션
rfind -dryrun true ./data

# 2단계: results.txt 검토
less results.txt

# 3단계: 문제 없으면 fdupes로 대화형 삭제 (더 세밀한 선택 가능)
fdupes -rd ./data
```

두 도구 모두 삭제 옵션은 되돌릴 수 없으니, 실행 전 반드시 -dryrun이나 목록 출력으로 먼저 확인하는 것을 권장합니다.